Національний університет «одеська політехніка»
Міністерство освіти і науки україни
Інститут комп’ютерних систем
Кафедра інформаційних систем

Протокол лабораторної роботи №1

з дисципліни: «Обчислювальні методи»

Тема: «Розгортання середовища та порівняльний аналіз мовних моделей»

Виконала:

студентка групи AI-245

Жиган Д. О.

Перевірили:

ас. Баранов О.М.

Одеса 2026

Мета й очікуваний результат

Після роботи студент повинен:

запустити мовну модель локально й керувати нею з коду;

розуміти, що локальна модель і хмарна — це один і той самий інтерфейс, і вміти перемикатися між ними зміною двох рядків;

вимірювати продуктивність інференсу й пояснювати, з чого складається затримка;

емпірично показати вплив temperature і top_p на стабільність відповідей.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

У першому завданні було проведено порівняння двох мовних моделей: qwen3:4b (міркувальна модель) та llama3.2:3b (інструктивна модель). Для виконання запитів використовувалася програма Ollama через PowerShell.

Спочатку обом моделям було поставлено однакове фактичне питання: «Столиця України? Відповідай одним словом». Після отримання відповідей було визначено кількість згенерованих токенів, загальний час виконання запиту та швидкість генерації токенів.

Таблиця 1 – Результати порівняння моделей на фактичному питанні

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ.
eval_count	187	5
Час, с	2,618	1,093
Токенів/с	178,32	193,30

Обидві моделі надали правильну відповідь. Проте модель qwen3:4b використала значно більше токенів через додаткові міркування, тоді як llama3.2:3b відповіла швидше та з меншими витратами ресурсів.

Далі було проведено перевірку здатності моделей розв'язувати логічні задачі. Для цього кожній моделі п'ять разів було поставлено задачу про вартість

ручки та зошита, загальна ціна яких становить 110 грн, а різниця у вартості — 100 грн. Правильна відповідь становить 5 грн.

Таблиця 2 – Результати порівняння моделей під час розв'язання логічної задачі

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5	1
eval_count	1478,6	156,2
Час, с	8,515	0,7134

Примітка. Під час третього запуску моделі llama3.2:3b виникла помилка генерації, тому цей запуск було зараховано як невдалий.

За результатами перевірки модель qwen3:4b правильно розв'язала задачу в усіх п'яти запусках, тоді як llama3.2:3b надала одну правильну відповідь. Міркувальна модель продемонструвала вищу точність, проте потребувала більше часу та токенів для формування відповідей.

Відповіді на запитання до завдання 1

1.Під час порівняння міркувальної моделі (qwen3:4b) та інструктивної (llama3.2:3b) на простому фактичному питанні відповідь обидві дали однакову, але міркувальна згенерувала в десятки разів більше токенів. Куди пішли ці токени?

Додаткові токени модель qwen3:4b (187 токенів проти 5 у llama3.2:3b) витратила на генерацію внутрішнього ланцюжка міркувань у прихованому полі thinking. Навіть для простого факту вона виконала самоперевірку, перебираючи можливі варіанти, перш ніж записати кінцеве слово «Київ» у поле response.

2.На яких типах задач міркувальні моделі дають перевагу над інструктивними, а на яких їх використання невиправдане?

Міркувальні моделі мають суттєву перевагу на задачах із багатоетапною логікою, математичними обчисленнями, програмуванням та аналізом, де проміжні кроки допомагають уникнути помилок (як у задачі про ручку й зошит, де qwen3:4b дала 5/5 правильних відповідей проти 1/5 у llama3.2:3b). На простих фактичних

запитах або базовій класифікації їх використання невиправдане через зайві витрати часу та обчислювальних ресурсів.

3.Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Інструктивні моделі проходять навчання з підкріпленням на основі людських відгуків (RLHF/SFT) для прямого та точного виконання команд користувача. Міркувальні моделі додатково навчаються методом навчання з підкріпленням на задачах із перевірюваним результатом (RLVR), що стимулює їх будувати довгі ланцюжки роздумів (Chain-of-Thought) і перевіряти власні гіпотези перед відповіддю.

4.Чи означає більша кількість згенерованих токенів у міркувальної моделі, що вона «розумніша» за інструктивну?

Ні, більша кількість токенів означає лише те, що модель витрачає більше обчислювального часу на етапі інференсу (test-time compute) для покрокової перевірки думок. На простих фактах це створює надлишкові витрати, а при відсутності базових знань модель може генерувати довгі, але хибні міркування.

5.Як пояснити різницю у швидкості генерації (токенів за секунду) між моделями qwen3:4b та llama3.2:3b?

Швидкість генерації (токенів/с) визначається кількістю параметрів моделі, її архітектурою та пропускною здатністю пам'яті обладнання. Оскільки llama3.2:3b має менше параметрів (3 млрд проти 4 млрд у qwen3:4b), для обчислення кожного наступного токена їй потрібно менше операцій, тому її швидкість вища (193,30 токенів/с проти 178,32 токенів/с).

Власні думки (особливості використання міркувальних та інструктивних моделей)

Використання міркувальних моделей виправдане тоді, коли ціна помилки висока, а задача вимагає логічних висновків або математики — навіть ціною збільшення часу очікування в кілька разів. Для швидких довідкових запитів, перекладу чи діалогів у реальному часі оптимальніше використовувати інструктивні моделі, оскільки вони не витрачають ресурси на зайву самоперевірку.

Висновок за завдання 1

У ході виконання завдання було досліджено особливості роботи міркувальної та інструктивної мовних моделей. Встановлено, що для простих фактичних питань інструктивна модель є швидшою та економнішою за кількістю токенів. Водночас під час розв'язання логічної задачі міркувальна модель показала вищу точність завдяки додатковим обчисленням. Отже, вибір моделі залежить від складності завдання та вимог до швидкості й точності відповіді.

Завдання 2. Вплив system-повідомлення на поведінку моделей

У другому завданні було досліджено вплив системного повідомлення на зміст, обсяг, мову та формат відповідей локальних мовних моделей llama3.2:3b і qwen3:4b.

Для виконання завдання відповідно до варіанта 5 використовувався запит: «Поясни різницю між масивом і зв'язним списком».

Для кожної моделі було використано два системні повідомлення:

«Ти лаконічний технічний асистент».

«Відповідай лише українською, максимум двома реченнями».

Для автоматизації дослідження використовувалася програма hello.py мовою Python. Кожна модель отримувала однаковий запит із двома різними системними повідомленнями. Після виконання програма визначала довжину відповіді, приблизну кількість речень та витрати токенів.

Таблиця 3 – Вплив system повідомлення

Модель	System	Довжина відповіді,символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	Лаконічний асистент	1869	Змішана, з помилками	16	Ні
llama3.2:3b	українською,2 речення	344	Українська з помилками	2	Частково
qwen3:4b	Лаконічний асистент	331	Українська	5	Так
qwen3:4b	українською,2 речення	282	Українська	2	Так

Під час використання першого системного повідомлення модель llama3.2:3b сформувала надмірно довгу відповідь, що не відповідало вимозі лаконічності. Крім того, у тексті були помилки у поясненні структур даних та некоректні мовні конструкції.

Після застосування другого системного повідомлення llama3.2:3b скоротила відповідь до двох речень, однак припустилася мовних і змістових помилок. Зокрема, модель неправильно пояснила доступ до елементів зв'язного списку через індекс.

Модель qwen3:4b продемонструвала краще дотримання обох інструкцій. Під час першого запуску вона надала коротке порівняння структур даних, а під час другого сформувала відповідь українською мовою з двох речень.

Таблиця 4 – Витрати токенів за різних system-повідомлень

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	Лаконічний асистент	54	571	625
llama3.2:3b	українською,2 речення	58	97	155
qwen3:4b	Лаконічний асистент	57	2569	2626

qwen3:4b	українською,2 речення	66	1059	1125

За результатами вимірювань встановлено, що обмеження кількості речень зменшило довжину відповідей обох моделей.

При цьому qwen3:4b використала значно більше вихідних токенів, ніж llama3.2:3b, хоча її видимі відповіді були коротшими. Це може бути пов'язано з особливостями внутрішнього генерування токенів міркувальною моделлю.

Проведений експеримент показав, що системні повідомлення суттєво впливають на обсяг і форму відповідей мовних моделей.

Модель llama3.2:3b недостатньо точно виконала вимогу лаконічності та допустила помилки у використанні української мови. Модель qwen3:4b продемонструвала краще дотримання встановлених обмежень і надала змістовніші пояснення.

Отже, чітке формулювання системного повідомлення допомагає керувати поведінкою мовної моделі, але не гарантує повної правильності її відповіді.

Відповіді на контрольні запитання до завдання 2

1.Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Модель qwen3:4b точніше виконала інструкцію, надавши коректну відповідь українською мовою рівно у двох реченнях. Модель llama3.2:3b також вклалася у ліміт двох речень, проте припустилася граматичних і змістових помилок.

2.Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Ні, після додавання прямої вказівки «Відповідай лише українською...» обидві моделі сформували відповідь українською мовою, хоча в тексті llama3.2:3b траплялися спотворені слова та невдалі кальки.

3.Як змінився prompt_tokens при зміні system-повідомлення і чому?

У моделі llama3.2:3b кількість prompt_tokens зросла з 54 до 58, а в qwen3:4b - з 57 до 66. Це відбулося через те, що новий системний промпт містить більше слів і символів, які при токенізації розбилися на більшу кількість токенів.

4.Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Оскільки qwen3:4b є міркувальною моделлю, у поле completion_tokens зараховуються не лише символи видимого тексту, а й усі внутрішні токени роздумів (thinking), витрачені на перевірку умов задачі та формулювання двох речень українською.

Відповіді на запитання до чат-бота

1.Під час порівняння інструктивної моделі (llama3.2:3b) та міркувальної (qwen3:4b) з різними system-повідомленнями виявлено різну точність виконання обмежень. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Різниця зумовлена обсягом і якістю даних для інструктивного донавчання (SFT/RLHF) та здатністю моделі утримувати обмеження під час генерації. Міркувальна модель qwen3:4b перед виводом тексту явно перевіряє у внутрішніх роздумах, чи відповідає її чернетка вимогам мови та кількості речень, тоді як мала llama3.2:3b генерує текст одразу і часто «збивається».

2.Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

Технічно обидва повідомлення потрапляють у спільне контекстне вікно, але розділяються спеціальними службовими токенами ролей (system і user). Під час навчання моделі спеціально вчать надавати ролі system вищий пріоритет як глобальним правилам поведінки, хоча в малих моделей ця ієрархія може порушуватися.

3.Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

У стандартному форматі API показник completion_tokens охоплює всю генерацію моделі - як прихований блок роздумів (thinking), так і фінальну відповідь користувачеві. Тому навіть для відповіді з 2 речень (282 символи) qwen3:4b витратила 1059 токенів проти 97 у llama3.2:3b.

4.Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Більшість сучасних моделей спроєктовані так, щоб дотримуватися system-інструкції, оскільки вона визначає базові обмеження безпеки та формату (ієрархія інструкцій). Проте невеликі локальні моделі (наприклад, на 3 млрд параметрів) через слабшу увагу до контексту часто піддаються на свіжіший запит із ролі user.

5.Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

У навчальних корпусах малих моделей частка україномовних текстів значно менша, ніж англомовних, а обмежена кількість параметрів не дозволяє однаково якісно зберігати граматичні та термінологічні зв'язки для всіх мов. У результаті модель може підмішувати іншомовні токени або робити кальки, особливо під час технічних пояснень.

Власні думки (про роль system-повідомлення)

System-повідомлення є зручним і швидким способом задати роль, мову та стиль відповіді, проте на малих моделях воно не дає 100% гарантії дотримання жорстких форматних обмежень. Якщо модель ігнорує інструкцію, варто додати в промпт конкретні приклади бажаного виводу (few-shot prompting), знизити temperature або застосувати програмну валідацію схеми відповіді (наприклад, через Pydantic чи Structured Outputs) з повторним запитом у разі помилки.

Висновок до завдання 2

У ході виконання другого завдання було досліджено вплив системних повідомлень на відповіді мовних моделей llama3.2:3b та qwen3:4b.

Встановлено, що використання конкретних інструкцій щодо мови та кількості речень дозволяє значно скоротити обсяг згенерованого тексту. Найкраще дотримання системних повідомлень продемонструвала модель qwen3:4b.

Також було виявлено, що виконання формальних вимог не завжди означає правильність змісту відповіді. Таким чином, системні повідомлення є важливим інструментом керування мовними моделями, однак результати їхньої роботи потребують додаткової перевірки.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Під час виконання третього завдання було проведено порівняння локальної моделі llama3.2:3b та хмарної моделі openai/gpt-oss-120b. Для цього було створено файли providers.py та compare.py, за допомогою яких здійснювалося підключення до моделей і надсилання однакового запиту.

Для порівняння було використано питання п'ятого варіанта: «Поясни різницю між масивом і зв'язним списком. Коротко». Під час виконання програми було визначено час формування відповіді, кількість вихідних токенів, довжину тексту, мову відповіді та наявність фактичних помилок.

Таблиця 5 – Порівняння локальної та хмарної мовних моделей

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	openai/gpt-oss-120b
Час відповіді, с	3,16	1,20
completion_tokens	177	477
Довжина відповіді, символів	577	1109
Мова відповіді	Змішана, з помилками	Українська
Фактичних помилок у відповіді	2	0

За результатами дослідження встановлено, що хмарна модель openai/gpt-oss-120b сформувала відповідь за 1,20 с, тоді як локальна модель llama3.2:3b витратила 3,16 с.

Хмарна модель надала більш повну та структуровану відповідь українською мовою, правильно пояснивши особливості масиву та зв'язного списку. Локальна модель допустила неточності у визначеннях і використовувала змішані мовні конструкції.

Отже, у проведеному експерименті хмарна модель продемонструвала вищу швидкість і якість відповіді, незважаючи на використання мережевого підключення.

Відповіді на контрольні запитання

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Безпосередньо в програмі порівняння не довелося змінювати жодного рядка, оскільки параметри моделей були винесені до файлу providers.py. Перемикання здійснювалося шляхом вибору відповідного провайдера.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Повнішою та точнішою виявилася відповідь хмарної моделі, оскільки вона правильно описала структуру масиву, зв'язного списку та складність операцій. Наприклад, хмарна модель зазначила, що доступ до елемента масиву за індексом має складність O(1), а до елемента зв'язного списку - O(n).

3. Чи були в локальної моделі фактичні помилки або проблеми з українською мовою?

Так, локальна модель допустила неточності у визначеннях структур даних і використовувала змішані українські та іншомовні слова. Це погіршило зрозумілість і точність відповіді.

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит передається через інтернет?

Хмарна модель працює на потужному серверному обладнанні, оптимізованому для швидкої обробки запитів. Завдяки цьому час обчислень може бути значно меншим, ніж на локальному комп'ютері, навіть з урахуванням мережевої затримки.

Відповіді на запитання до чат-бота

1.Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

OpenAI однією з перших випустила зручний і простий REST API для чат-моделей, навколо якого швидко сформувалася величезна екосистема бібліотек і фреймворків. Іншим провайдерам та локальним серверам (як-от Ollama чи Groq) вигідно підтримувати цей самий інтерфейс, щоб розробники могли інтегрувати їхні моделі без переписування коду клієнта.

2.При порівнянні локальної моделі на 3 млрд параметрів і хмарної на 70 млрд (у проведеному досліді — 120 млрд) зафіксовано суттєву різницю в результатах. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Більша кількість параметрів дає моделі значно ширшу «пам'ять» для зберігання фактів, краще розуміння мовних нюансів і складніших логічних зв'язків, що й забезпечило перевагу хмарної моделі. Водночас важливу роль відіграють якість навчальних даних, архітектура (наприклад, MoE) та етапи пост-навчання (RLHF), завдяки яким добре натренована середня модель може перевершувати більшу, але гірше навчену.

3.Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Локальний комп'ютер виконує інференс на звичайному процесорі або споживчій відеокарті з обмеженою пропускною здатністю оперативної пам'яті, що обмежує швидкість генерації. Хмарні провайдери використовують кластери промислових прискорювачів і оптимізовані рушії інференсу, де генерація займає частки секунди, що значно менше за виграш локальної моделі від відсутності мережевої затримки.

4.Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

При зверненні до хмарного API всі вхідні дані та відповіді передаються на сервери стороннього провайдера, де вони можуть логуватися або використовуватися для донавчання моделей, що створює ризик витоку конфіденційної інформації чи персональних даних. У комерційних проєктах для захисту укладають угоди про нульове збереження даних (Zero Data Retention), знеособлюють текст перед відправкою (PII masking) або розгортають моделі в ізольованому приватному контурі.

5.Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Rate limit - це технічне обмеження провайдера на максимальну кількість запитів (RPM) або токенів (TPM) за хвилину чи добу, створене для захисту серверів від перевантаження. Розробники справляються з цим за допомогою черг запитів, кешування однакових відповідей, алгоритму повторних спроб із експоненційною затримкою (exponential backoff) або автоматичного перемикання на резервного провайдера.

Власні думки (вибір між локальною та хмарною моделями)

Локальну модель доцільно обирати для роботи з конфіденційними документами, персональними даними, внутрішнім кодом або в умовах відсутності чи нестабільності інтернету, а також для масової фонової обробки простих текстів

без витрат на API. Хмарну модель краще використовувати для складних інтелектуальних завдань, генерації якісного україномовного контенту, глибокого аналізу та інтерактивних сервісів, де критично важливі висока точність і швидкість відповіді.

Висновок за завданням 3

У ході виконання третього завдання було проведено порівняння локальної та хмарної мовних моделей із використанням єдиного програмного інтерфейсу. Встановлено, що хмарна модель openai/gpt-oss-120b надала повнішу та точнішу відповідь за менший час, ніж локальна llama3.2:3b.

На мою думку, локальні моделі доцільно використовувати для роботи з конфіденційною інформацією, за відсутності інтернету або коли важлива незалежність від зовнішніх сервісів. Хмарні моделі краще підходять для складних завдань, які потребують високої якості відповідей і швидкої обробки даних.

Таким чином, використання єдиного інтерфейсу спрощує перемикання між провайдерами та дозволяє обирати модель відповідно до вимог конкретного завдання.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

У четвертому завданні було проведено дослідження швидкості роботи локальної мовної моделі llama3.2:3b та хмарної моделі openai/gpt-oss-120b. Для виконання завдання використовувалася програма мовою Python, яка надсилала запити до моделей через єдиний програмний інтерфейс.

Метою дослідження було визначення часу до появи першого токена (TTFT), загального часу формування відповіді та швидкості генерації тексту в символах за секунду.

Відповідно до варіанта 5 для проведення експерименту було використано три запити різної довжини:

Короткий запит — «Столиця Єгипту?».

Середній запит — «Поясни в одному абзаці, що таке рекурсія».

Довгий запит — «Напиши інструкцію з 10 пунктів, як почати працювати з Git».

Для проведення вимірювань було створено файл benchmark.py. Програма використовувала потокове отримання відповіді, що дозволило визначити час появи першого текстового фрагмента. Перед початком вимірювань виконувався прогрів моделей, після чого кожен запит запускався тричі без паралельного виконання.

Для підвищення точності отриманих результатів обчислювалися медіанні значення трьох запусків. Результати було збережено у файлі results.csv.

Таблиця 6 – Результати вимірювання швидкості мовних моделей

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2,023	2,086	13,4
локальна	llama3.2:3b	середній	2,022	2,563	122,7
локальна	llama3.2:3b	довгий	2,040	5,000	385,6
хмарна	openai/gpt-oss-120b	короткий	0,274	0,308	73,5
хмарна	openai/gpt-oss-120b	середній	0,528	0,679	514,0
хмарна	openai/gpt-oss-120b	довгий	0,556	2,409	1134,1

За результатами вимірювань встановлено, що хмарна модель openai/gpt-oss-120b продемонструвала менший час до появи першого токена та вищу швидкість генерації відповідей для всіх трьох типів запитів.

Для короткого запиту загальний час відповіді локальної моделі становив 2,086 с, тоді як хмарної - 0,308 с. При виконанні довгого запиту час збільшився до 5,000 с та 2,409 с відповідно. Таким чином, хмарна модель виявилася швидшою, хоча відносна різниця за загальним часом виконання зменшилася при переході до довгого запиту.

Додатково було проведено порівняння холодного та теплого старту локальної моделі llama3.2:3b. Для цього модель було зупинено за допомогою Ollama, після чого виконано перший запит і виміряно його тривалість. Наступний запит виконувався без повторної зупинки моделі.

Для вимірювання використовувався файл start_test.py, у якому було реалізовано фіксацію часу до появи першого токена та загального часу відповіді.

Таблиця 7 – Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	3,618	2,024
Всього, с	3,679	2,331

За результатами дослідження встановлено, що холодний старт потребує більше часу, ніж теплий. Це пояснюється необхідністю завантаження моделі в оперативну або відеопам'ять під час першого запиту. При повторному зверненні модель уже знаходиться в пам'яті, тому початок генерації відбувається швидше.

Для наочного порівняння результатів було побудовано стовпчикову діаграму засобами бібліотеки Matplotlib. На діаграмі відображено швидкість генерації відповідей у символах за секунду для шести комбінацій моделей та типів запитів.

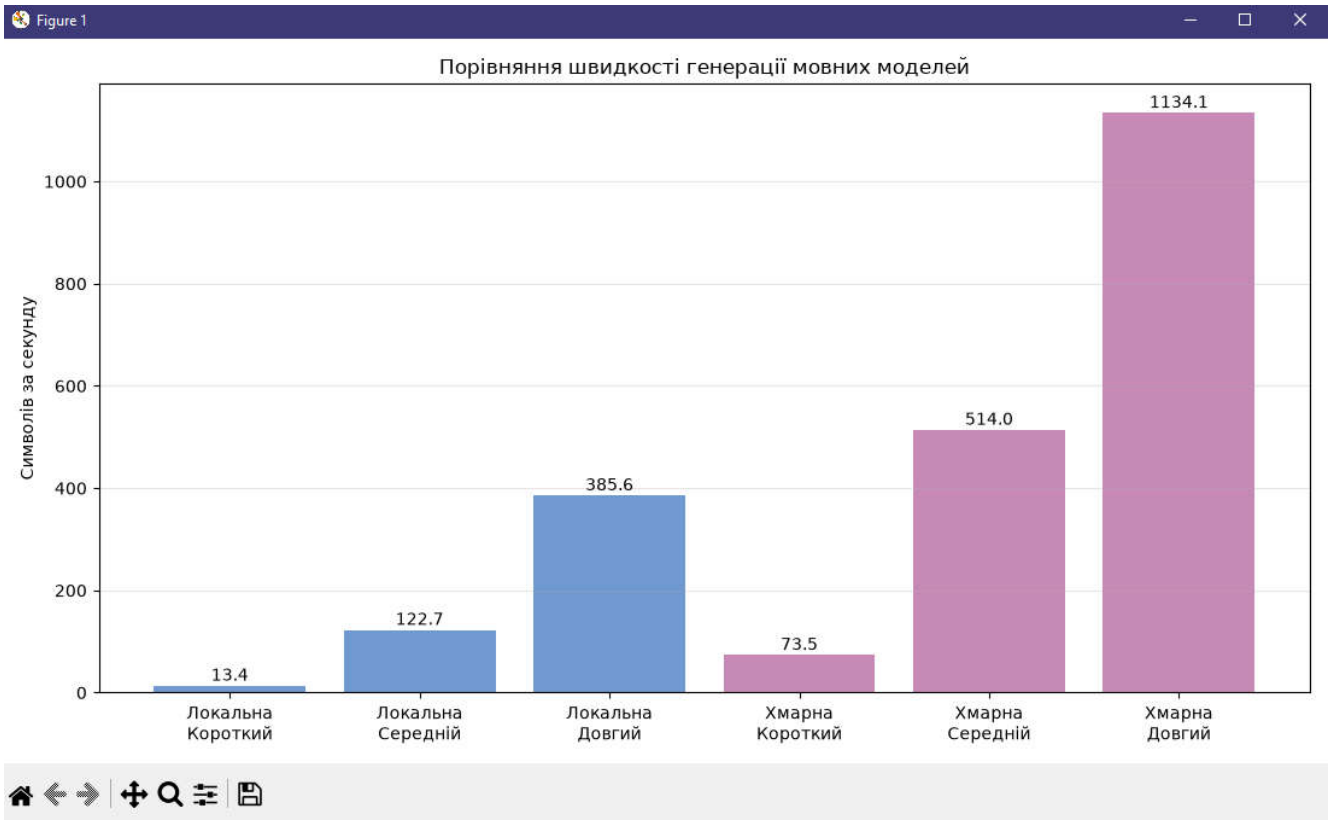


Рисунок 1 – Порівняння швидкості генерації відповідей локальної та хмарної мовних моделей

З отриманої діаграми видно, що хмарна модель забезпечила вищу швидкість генерації тексту для кожного типу запиту. Найбільше значення швидкості становило 1134,1 символа за секунду для хмарної моделі під час виконання довгого запиту, тоді як локальна модель досягла 385,6 символа за секунду.

Відповіді на контрольні запитання

1.У якої моделі кращий TTFT і чому саме в неї?

Кращий показник TTFT продемонструвала хмарна модель openai/gpt-oss-120b (0,274–0,556 с проти 2,022–2,040 с у локальної llama3.2:3b), оскільки потужні серверні прискорювачі обробляють вхідний промпт (prefill) у рази швидше за локальний комп'ютер, навіть з урахуванням затримки мережі.

2.Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — у чому причина?

У проведеному експерименті обидва показники збіглися на користь хмарної моделі, проте на практиці вони часто не збігаються, оскільки TTFT залежить від мережевого пінгу та швидкості паралельної обробки вхідного запиту, а швидкість генерації (симв./с) - від пропускної здатності пам'яті при послідовному створенні нових токенів.

3.Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

На короткому промпті хмарна модель була швидшою за загальним часом майже в 6,8 раза (0,308 с проти 2,086 с), а на довгому - приблизно в 2,1 раза (2,409 с проти 5,000 с), оскільки в локальної моделі висока початкова затримка (TTFT ≈ 2 с) на довгій відповіді «розчинилася» у загальному часі генерації.

4.Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Повтори помітно відрізнялися через коливання фонового навантаження на систему та мережу (наприклад, на довгому промпті час локальної моделі коливався від 3,990 с до 5,687 с), тому одного запуску недостатньо - вибірка з кількох прогонів і медіана необхідні для відсіювання випадкових викидів.

5.Чому холодний старт повільніший за теплий?

Під час холодного старту система витрачає додатковий час (у досліді TTFT зріс із 2,024 с до 3,618 с) на зчитування файлу ваг моделі з диска та завантаження його в оперативну чи відеопам'ять, тоді як при теплому старті модель уже перебуває в пам'яті й готова до обчислень миттєво.

Відповіді на запитання до чат-бота

1.Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Під час TTFT запит передається по мережі (для хмари), токенізується, а модель виконує фазу prefill - паралельно обчислює стани уваги (KV-кеш) для всіх вхідних токенів запиту, щоб передбачити найперший вихідний токен. Цей показник вимірюють окремо, бо він визначає сприйняття «чуйності» інтерфейсу користувачем (час очікування початку відповіді), тоді як швидкість генерації впливає на тривалість отримання великого тексту.

2.Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Хмарна модель може мати вищий TTFT через мережеву затримку на встановлення з'єднання, передачу пакетів через інтернет та можливе очікування в черзі на сервері провайдера. Проте щойно обчислення починаються, промислові GPU генерують подальші токени зі швидкістю в сотні токенів за секунду, значно випереджаючи локальний процесор.

3.Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Холодний старт - це звернення до моделі, яка ще не завантажена в RAM або VRAM (або була вивантажена після простою). Перший запуск вимагає копіювання гігабайтів параметрів із накопичувача (SSD/HDD) у пам'ять та ініціалізації обчислювальних буферів, тоді як наступні запити використовують уже завантажену модель (теплий старт).

4.Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

На час виконання впливають нестабільні зовнішні фактори: фонові процеси ОС, перемикання частот процесора, коливання мережевого пінгу чи затримки на боці API. Медіана (на відміну від одного заміру чи навіть середнього арифметичного) стійка до поодиноких аномальних затримок і показує типову, реальну швидкість роботи системи.

5.Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Усі токени вхідного запиту відомі заздалегідь, тому графічний або центральний процесор може обробити їх паралельно за один прохід матричних обчислень (compute-bound фаза *prefill*). Генерація відповіді (фаза *decode*) є авторегресійною: кожен наступний токен залежить від попереднього, тому вони обчислюються суворо по одному за крок, що впирається у пропускну здатність пам'яті.

Власні думки (що важливіше для чат-бота vs фонової обробки)

Для інтерактивного чат-бота критично важливим є низький TTFT, оскільки користувач у режимі стрімінгу починає читати перші рядки майже одразу і не відчуває «зависання» системи. Натомість для фонової пакетної обробки великих обсягів тексту (наприклад, сумаризації тисяч документів) TTFT не має вирішального значення - там головним показником є максимальна швидкість генерації (симв./с) і мінімальний загальний час виконання всього обсягу задач.

Висновок до завдання 4

У ході виконання четвертого завдання було досліджено швидкість роботи локальної та хмарної мовних моделей залежно від довжини запиту. Було визначено час до появи першого токена, загальний час формування відповіді та швидкість генерації символів.

За результатами експерименту встановлено, що хмарна модель openai/gpt-oss-120b продемонструвала кращі показники швидкості для короткого, середнього та довгого запитів. Також було підтверджено, що холодний старт локальної моделі повільніший за теплий через необхідність завантаження моделі в пам'ять комп'ютера.

Отримані результати дозволили зробити висновок, що для інтерактивних систем важливо мінімізувати затримку до першого токена, а для обробки великих

обсягів інформації = забезпечувати високу швидкість генерації та мінімальний загальний час виконання запиту.

Завдання 5. Оцінювання якості моделей за рубрикою

У п'ятому завданні було проведено порівняльне оцінювання якості відповідей трьох мовних моделей: локальних llama3.2:3b і qwen3:4b та хмарної openai/gpt-oss-120b.

Метою дослідження було визначення здатності моделей правильно відповідати на фактичні запитання, дотримуватися заданого формату, стискати текст без втрати інформації, генерувати програмний код, виконувати логічні обчислення та формувати грамотні відповіді українською мовою.

Відповідно до варіанта 5 «Фінанси» було підготовлено шість тестових завдань:

1. Фактичні знання - визначити рік уведення гривні як національної валюти України.
2. Дотримання формату - сформувати JSON з полями currency, code та year.
3. Стиснення тексту - скоротити текст про історію гривні до двох речень зі збереженням усіх числових значень.
4. Генерація коду - написати функцію compound_interest(principal, rate, years) для обчислення складних відсотків.
5. Логічне мислення - визначити кінцеву суму депозиту 20 000 грн під 12% річних з урахуванням податку 19,5% на відсотки.
6. Якість української мови - пояснити відмінність між акцією та облігацією двома абзацами.

Для автоматизації перевірки було створено програму quality.py мовою Python. Кожне завдання виконувалося двічі для кожної моделі з параметром temperature = 0,7. Загалом було отримано 36 відповідей, які зберігалися у файлі quality.json.

Оцінювання проводилося за трибальною шкалою: 0 балів - завдання не виконано; 1 бал - завдання виконано частково; 2 бали - завдання виконано повністю. Максимальна оцінка для однієї моделі становила 12 балів.

Таблиця 8 – Оцінка якості моделей за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	0	0	2
2. Формат	2	2	2
3. Стиснення	0	2	2
4. Код	2	2	1
5. Логіка	0	2	2
6. Українська	0	1	1
Разом /12	4	9	10

За результатами оцінювання найвищий бал отримала хмарна модель openai/gpt-oss-120b - 10 із 12 можливих. Модель qwen3:4b набрала 9 балів, а llama3.2:3b - 4 бали.

Найбільші відмінності спостерігалися під час перевірки фактичних знань, стиснення тексту та логічного мислення. Модель llama3.2:3b допускала значні помилки у фактах, втрачала числові дані та неправильно виконувала обчислення.

У завданні на генерацію коду локальні моделі правильно реалізували необхідну функцію. Хмарна модель також сформувала правильний алгоритм, однак додала Markdown-обгортку, хоча за умовою вимагалося надати лише код.

Для перевірки стабільності моделей було порівняно оцінки, отримані за результатами двох запусків кожного завдання.

Таблиця 9 – Розбіжність оцінок між двома запусками

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	Ні	Ні	Ні
2. Формат	Ні	Ні	Ні
3. Стиснення	Ні	Ні	Ні

4. Код	Ні	Ні	Ні
5. Логіка	Ні	Ні	Ні
6. Українська	Ні	Ні	Ні

Порівняння показало, що оцінки двох запусків за встановленою рубрикою не відрізнялися. Проте самі відповіді іноді були різними. Наприклад, у фактичному завданні модель llama3.2:3b під час першого запуску вказала 1992 рік, а під час другого - 1961 рік. Обидві відповіді були неправильними, тому отримали однакову оцінку - 0 балів.

Це свідчить про те, що однакові оцінки не завжди означають повну стабільність відповідей мовної моделі.

Приклад провалу мовної моделі

Для детальнішого аналізу було обрано помилку локальної моделі llama3.2:3b у завданні на перевірку фактичних знань.

Модель: llama3.2:3b.

Повний текст запиту:

«У якому році гривню ввели як національну валюту України? Відповідай лише роком.»

Повна відповідь першого запуску:

1992.

Повна відповідь другого запуску:

1961

Правильна відповідь:

1996 рік.

Правильність відповіді перевірено за інформацією Національного банку України щодо історії запровадження гривні.

Пояснення помилки

Модель двічі надала неправильну відповідь, причому вказала різні роки. Це демонструє недостатню надійність її фактичних знань. Подібне явище називають галюцинацією мовної моделі, коли вона формує переконливу на вигляд відповідь, яка не відповідає дійсності.

Також показовою виявилася задача на логічне мислення, у якій необхідно було розрахувати суму депозиту з урахуванням податку.

Правильний розрахунок:

Нараховані відсотки: 20 000 × 0,12 = 2 400 грн.

Податок: 2 400 × 0,195 = 468 грн.

Чистий дохід: 2 400 − 468 = 1 932 грн.

Кінцева сума: 20 000 + 1 932 = 21 932 грн.

Модель llama3.2:3b під час другого запуску отримала неправильний результат - 17 632 грн, тоді як qwen3:4b і openai/gpt-oss-120b правильно визначили кінцеву суму 21 932 грн.

Відповіді на контрольні запитання

1.Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Найбільше балів набрала хмарна модель openai/gpt-oss-120b (10/12), випередивши qwen3:4b (9/12) та llama3.2:3b (4/12). Основна різниця виникла на завданнях із фактичних знань (де впоралася лише хмарна модель), а також у стисненні тексту зі збереженням чисел і багатоетапній логіці (де llama3.2:3b отримала 0 балів).

2.На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Оцінки в балах між двома прогонами не розійшлися на жодному завданні, однак усередині прогонів відповіді відрізнялися (наприклад, llama3.2:3b у завданні 1 назвала дві різні хибні дати - 1992 і 1961). Це свідчить про те, що при незнанні

факту мала модель нестабільно галюцинує, і без перевірки першоджерела повторні прогони можуть створювати ілюзію стабільності помилки.

3.Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Так, у завданні 2 усі три моделі повернули чистий JSON без markdown-обгортки (що успішно пройшло перевірку через json.loads()), хоча у завданні 4 («лише код») хмарна модель усе ж додала markdown-оформлення, через що втратила 1 бал.

4.Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Так, llama3.2:3b отримала 0 балів через грубі мовні спотворення, кальки та неприродні конструкції, а qwen3:4b і openai/gpt-oss-120b отримали по 1 балу через окремі стилістично кострубаті звороти та русизми у фінансовій термінології.

Відповіді на запитання до чат-бота

1.При оцінюванні мовних моделей за рубрикою з шести завдань і трибальною шкалою отримано оцінки 4/12, 9/12 та 10/12. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

На практиці широко застосовують стандартизовані бенчмарки (MMLU, GSM8K, HumanEval для коду), автоматизоване оцінювання «LLM-as-a-Judge» (коли потужніша модель оцінює відповіді слабшої за заданими критеріями), а також сліпе парне порівняння людьми (наприклад, Chatbot Arena з рейтингом Elo).

2.Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Під час попереднього навчання та SFT моделі бачили мільйони прикладів технічної документації, де код і JSON майже завжди оформлені в блоки ```json. Цей статистичний патерн настільки сильний, що в процесі авторегресійної генерації він часто переважує текстову заборону користувача.

3.Чому малі мовні моделі гірше пишуть українською, ніж англійською?

В інтернет-корпусах, на яких тренуються моделі, частка англійської мови становить переважну більшість, тоді як українська представлена в рази менше (і часто поступається навіть російській, що спричиняє кальки). Крім того, мала кількість параметрів і менш ефективний словник токенізатора для кирилиці обмежують здатність моделі формувати природні українські конструкції.

4.Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

При ненульовій температурі (у проведеному тесті temperature=0.7) вибір кожного наступного токена відбувається ймовірнісним методом семплінгу, а не детерміновано. Через авторегресійну природу генерації випадковий вибір навіть одного іншого слова на початку відповіді змінює весь подальший контекст і може привести до іншого результату або помилки.

5.Що таке галюцинація мовної моделі і чому модель радше вигадає факт, ніж визнає незнання?

Галюцинація - це генерація впевненої на вигляд, але фактично неправдивої інформації (як вигадані 1992 чи 1961 роки замість 1996). Мовна модель не має вбудованої бази перевірених фактів і механізму оцінки власної невпевненості - вона оптимізована передбачати найімовірніше продовження тексту за формою, тому генерує найбільш статистично схоже число.

Власні думки (оцінювання за рубрикою та вибір моделі для української мови)

Оцінювання за фіксованою рубрикою виявилося значно об'єктивнішим за суб'єктивне враження, оскільки дозволило чітко відділити якість формату від фактичної точності, а найпоказовішими стали завдання на регіональні факти, стиснення з числами та багатоетапну логіку. Для роботи з україномовними текстами найкраще підходить хмарна модель (openai/gpt-oss-120b), а серед

локальних рішень — qwen3:4b, яка показала майже такий самий високий рівень виконання завдань (9/12 проти 10/12).

Висновок до завдання 5

У ході виконання п'ятого завдання було проведено порівняльне оцінювання якості трьох мовних моделей за шістьма критеріями. Для кожного завдання було виконано два запуски, після чого відповіді перевірялися відповідно до встановленої рубрики.

За результатами дослідження найвищий бал отримала хмарна модель openai/gpt-oss-120b - 10 із 12 можливих. Модель qwen3:4b набрала 9 балів, а llama3.2:3b - 4 бали.

Встановлено, що локальні моделі можуть успішно генерувати програмний код і дотримуватися заданого формату, але водночас допускати помилки у фактичних знаннях та логічних обчисленнях. Також було виявлено недоліки у використанні української мови.

Отже, оцінювання за рубрикою дозволяє визначити сильні та слабкі сторони мовних моделей. Для отримання надійних результатів важливо проводити повторні запуски, перевіряти правильність відповідей і враховувати не лише загальний бал, а й особливості виконання окремих завдань.

Завдання 6. Керування випадковістю: temperature і top_p

У шостому завданні було досліджено вплив параметрів temperature і top_p на різноманітність, правильність та відтворюваність відповідей мовної моделі llama3.2:3b.

Метою дослідження було визначення залежності результатів генерації від налаштувань випадковості та встановлення найбільш придатних значень параметрів для творчих і фактичних завдань.

Для проведення експерименту відповідно до варіанта 5 було використано два запити:

Творче завдання: «Придумай назву застосунку для пошуку партнерів для тренувань. Відповідай лише назвою».

Фактичне запитання: «У якому році гривню ввели як національну валюту України? Відповідай лише роком».

Для автоматизації дослідження було створено програму sampling.py мовою Python. Творчий запит виконувався по п'ять разів для значень temperature 0,0; 0,7; 1,5 та значень top_p 0,1; 0,5; 1,0. Під час перевірки top_p температура залишалася сталою і дорівнювала 0,7.

Фактичне запитання виконувалося по п'ять разів при temperature 0,0 і 1,5. Додатково було виконано десять повторів фактичного запиту при temperature 0,0 для перевірки відтворюваності.

Таблиця 10 – Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	0
temperature	0.7	5	—
temperature	1.5	5	0
top_p	0.1	1	—
top_p	0.5	3	—
top_p	1.0	5	—

За результатами дослідження встановлено, що при temperature = 0,0 модель п'ять разів згенерувала однакову назву «ТренерPlus». При temperature = 0,7 було отримано п'ять різних назв, які переважно відповідали тематиці застосунку.

При temperature = 1,5 також було отримано п'ять унікальних відповідей, однак їхня якість погіршилася. Зокрема, модель створювала назви зі змішаними мовами та некоректними символами, наприклад «ФітNESS-пapnar.com» і «Схваćай!».

Під час зміни top_p спостерігалося поступове збільшення різноманітності відповідей. При top_p = 0,1 було отримано одну унікальну назву, при 0,5 - три, а при 1,0 - п'ять.

У фактичному завданні правильна відповідь становить 1996 рік (джерело: Національний банк України - https://bank.gov.ua/ua/uah). Проте при temperature = 0,0 модель п'ять разів відповіла «1992», тобто не надала жодної правильної відповіді. При temperature = 1,5 модель називала різні неправильні роки: 1987, 1938, 1948 та 1992.

Ці результати показують, що збільшення температури підвищує випадковість відповідей, але не гарантує їхньої правильності. Низька температура робить генерацію передбачуванішою, проте модель може стабільно повторювати фактичну помилку.

Для перевірки відтворюваності фактичний запит було виконано десять разів із параметром temperature = 0,0.

Таблиця 11 – Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0,0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	Так

Під час усіх десяти запусків модель надала однакову відповідь — «1992». Отже, в цьому експерименті temperature = 0,0 забезпечила повну повторюваність тексту відповіді, але не її фактичну правильність.

Відповіді на контрольні запитання

1.Яке значення температури доцільно встановити агенту, який має повертати JSON, і чому?

Для агента, що повертає JSON, доцільно встановити temperature = 0.0 (або 0.1), щоб мінімізувати випадковість вибору токенів і забезпечити максимально стабільну та передбачувану структуру даних без зайвих синтаксичних відхилень.

2.Яке значення температури підійде генератору варіантів заголовків?

Для генератора варіантів заголовків оптимальним є помірне значення temperature = 0.7, оскільки в експерименті воно дало максимальну різноманітність (5 унікальних назв із 5) зі збереженням зв'язності, тоді як при 1.5 з'являлися спотворені слова.

3.Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Temperature масштабує логіти всіх токенів словника, змінюючи контрастність розподілу ймовірностей (роблячи його гострішим або більш плоским), тоді як top_p не змінює співвідношення ймовірностей, а динамічно відсікає «хвіст» малоймовірних токенів, залишаючи лише найімовірніших кандидатів із сумарною ймовірністю p.

4.Чи означає temperature=0 повну відтворюваність результату? Що показав експеримент із 10 запусків?

Теоретично temperature=0 не гарантує 100% відтворюваності через особливості паралельних обчислень із рухомою комою на GPU/CPU, проте в проведеному експерименті з 10 запусків модель показала абсолютну повторюваність (1 унікальна відповідь «1992» усі 10 разів, хоча вона й була фактично хибною).

5.Яка найбезглуздіша відповідь з'явилася при temperature=1.5 і чому вона могла виникнути?

Найбезглуздішою стала назва «ФітNESS-napnar.com», у якій перемішалися кирилиця, латиниця та доменне розширення. Вона з'явилася через те, що висока

температура згладила розподіл ймовірностей і штучно підвищила шанси на вибір рідкісних, нетипових для українського тексту токенів.

Відповіді на запитання до чат-бота

1.Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?

Перед застосуванням функції softmax модель ділить сирі оцінки (логіти z_i) кожного токена на значення температури T: $P_i = \frac{\exp(z_i/T)}{\sum \exp(z_j/T)}$. При $T < 1$ різниця між логітами збільшується, концентруючи ймовірність на лідері, а при $T > 1$ різниця згладжується і розподіл наближається до рівномірного.

2.Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?

Temperature змінює форму всього розподілу ймовірностей для кожного слова у словнику, надаючи шанс навіть вкрай малоймовірним токенам. Top_p (nucleus sampling) сортує токени за спаданням імовірності та обрізає список кандидатів на межі, де їхня кумулятивна сума досягає порога p, повністю виключаючи абсурдний «хвіст» розподілу.

3.Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

На практиці рекомендується налаштовувати лише один із цих параметрів, залишаючи інший за замовчуванням (temperature=1.0 або top_p=1.0), оскільки їхня одночасна зміна ускладнює прогнозування поведінки моделі та унеможливлює точне визначення причини змін у генерації.

4.Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

При temperature=0 використовується жадібний пошук (greedy decoding), який завжди обирає токен із найвищою ймовірністю, але через неасоціативність

операцій із рухомою комою при паралельних обчисленнях на GPU та динамічне пакетування запитів (batching) у хмарних сервісах мікропохибки можуть змінити лідера при майже рівних імовірностях двох токенів.

5.Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Для витягу даних, класифікації, генерації коду та JSON на практиці використовують temperature в діапазоні 0.0–0.2 для максимальної точності та детермінованості. Для написання художніх текстів, мозкового штурму та генерації ідей зазвичай обирають значення 0.7–1.0, уникаючи значень вище 1.2, де текст втрачає зв'язність.

Власні думки (вибір параметрів для наступних лабораторних)

Для наступних лабораторних робіт, де модель виступатиме як компонент агента й повертатиме структуровані дані (JSON або виклики інструментів), оптимально обрати temperature = 0.0 (або 0.1) та top_p = 1.0. Налаштування за замовчуванням (зазвичай temperature = 0.7–1.0) для таких задач є непридатними, оскільки вносять зайву варіативність, підвищують ризик синтаксичних помилок у форматі та галюцинацій у значеннях полів.

Висновок до завдання 6

У ході виконання шостого завдання було досліджено вплив параметрів temperature і top_p на різноманітність, правильність та відтворюваність відповідей локальної мовної моделі llama3.2:3b.

Встановлено, що зі збільшенням temperature і top_p зростала кількість унікальних відповідей на творче завдання. При temperature = 0,7 було отримано п'ять різних назв застосунку, тоді як при temperature = 1,5 з'явилися некоректні мовні конструкції.

У фактичному завданні модель не надала жодної правильної відповіді ні при temperature = 0,0, ні при temperature = 1,5. Перевірка відтворюваності показала, що

за десять запусків із temperature = 0,0 було отримано лише одну унікальну відповідь, яка водночас була неправильною.

Отже, temperature і top_p дозволяють керувати випадковістю та різноманітністю генерації, але не гарантують точності інформації. Для структурованих і фактичних завдань доцільно використовувати низьку температуру, а для творчих - помірно підвищену.

Загальні висновки

Таблиця 12 – Підсумкове порівняння моделей

Критерій	Локальна мала (3B)	Локальна більша (4B/8B)	Хмарна
Швидкість генерації	13,4–385,6 симв./с	Не вимірювалася в завданні 4	73,5–1134,1 симв./с
TTFT	2,022–2,040 с	Не вимірювався в завданні 4	0,274–0,556 с
Якість, балів /12	4	9	10
Стабільність формату (завдання 2)	JSON правильний за синтаксисом, але з помилками в даних	JSON правильний за синтаксисом, але з помилками в даних	JSON правильний за синтаксисом і змістом
Потрібні ресурси	Ресурси власного комп'ютера, порівняно невеликі вимоги	Ресурси власного комп'ютера, більші вимоги, ніж у 3B	Серверні ресурси провайдера
Вартість	Без окремої оплати запитів, витрати на обладнання та електроенергію	Без окремої оплати запитів, витрати на обладнання та електроенергію	Залежить від тарифів і лімітів API
Куди йдуть ваші дані	Обробляються локально	Обробляються локально	Передаються зовнішньому провайдеру через інтернет

Працює без інтернету	Так, після завантаження моделі	Так, після завантаження моделі	Ні
Коли обирати саме її	Для простих локальних завдань із невисокими вимогами до якості	Для локальної роботи, коли важливі конфіденційність і краща якість	Для складних завдань, коли важливі швидкість, якість і точність

Підсумкове порівняння моделей

Після виконання експериментів було проведено загальне порівняння трьох мовних моделей за показниками швидкості, якості, стабільності формату, вимог до ресурсів, вартості та способу обробки даних.

За результатами оцінювання якості найкращий результат продемонструвала хмарна модель openai/gpt-oss-120b, яка набрала 10 із 12 балів. Локальна модель qwen3:4b отримала 9 балів, тоді як llama3.2:3b - 4 бали.

Під час вимірювання швидкості хмарна модель також показала кращі результати порівняно з llama3.2:3b. Вона мала менший час до появи першого токена та забезпечувала вищу швидкість формування відповідей для всіх досліджених типів запитів.

Водночас локальні моделі мають переваги, пов'язані з можливістю роботи без постійного підключення до інтернету та обробкою даних без передавання запитів зовнішнім сервісам.

На мою думку, локальну модель доцільно використовувати, коли важлива конфіденційність інформації, автономність або відсутність окремої оплати кожного запиту. Серед досліджених локальних моделей я обрала б qwen3:4b, оскільки вона продемонструвала значно кращу якість відповідей порівняно з llama3.2:3b.

Хмарну модель доцільніше обирати для складніших завдань, коли пріоритетом є якість відповідей та швидкість роботи, а передавання інформації зовнішньому провайдеру допускається умовами використання системи.

Отже, вибір мовної моделі повинен залежати від вимог конкретного завдання, доступних обчислювальних ресурсів, потреби в конфіденційності та очікуваної якості результатів.

Загальний висновок

У ході виконання лабораторної роботи було досліджено особливості розгортання, налаштування та використання локальних і хмарних мовних моделей. Для проведення експериментів використовувалися локальні моделі llama3.2:3b і qwen3:4b, запущені за допомогою Ollama, а також хмарна модель openai/gpt-oss-120b через програмний інтерфейс API.

Було проведено порівняння міркувальних та інструктивних моделей, досліджено вплив системних повідомлень на формування відповідей, а також проаналізовано відмінності між локальним і хмарним способами виконання запитів.

Під час дослідження продуктивності було виміряно час до появи першого токена, загальну тривалість відповіді та швидкість генерації тексту. Встановлено, що хмарна модель продемонструвала кращі показники швидкості порівняно з локальною llama3.2:3b.

Оцінювання якості відповідей за спеціальною рубрикою показало, що найвищий результат отримала хмарна модель openai/gpt-oss-120b — 10 із 12 балів. Модель qwen3:4b набрала 9 балів, а llama3.2:3b — 4 бали.

Також було досліджено вплив параметрів temperature і top_p на випадковість генерації. Встановлено, що збільшення цих параметрів може підвищувати різноманітність відповідей, однак не гарантує їхньої правильності. За низької температури модель демонструвала стабільні результати, але могла повторювати фактичні помилки.

У результаті виконання лабораторної роботи було отримано практичні навички налаштування мовних моделей, використання API, проведення експериментальних вимірювань, аналізу відповідей та оцінювання їхньої якості.

Зроблено висновок, що вибір між локальною та хмарною моделлю повинен здійснюватися з урахуванням вимог до швидкості, якості, обчислювальних ресурсів, конфіденційності даних і можливості автономної роботи.